Instituto Tecnológico de Costa Rica

Profesor: Ronny García Ramírez

Estudiantes:
Tomás Campos Uribe
Juan Pablo Cruz

Verificación funcional de circuitos integrados

Proyecto #1:

Fecha de entrega:
28/4/2026

Objetivo General

Desarrollar y aplicar un plan de verificación modular que permita estimular, observar y validar el comportamiento de una FIFO de manera sistemática

Objetivos Específicos

- Separar los diseños iniciales de algunos de los transactores para su debido funcionamiento a como se vió en clase.
- Implementar un conjunto de pruebas que abarque casos de esquina, secuencias aleatorias, pruebas de estrés y patrones de datos especiales, asegurando la cobertura de todos los escenarios críticos de operación.

Palabras clave

FIFO, Functional Verification, SystemVerilog, Testbench, Driver, Monitor, Scoreboard, Generator, Environment, Randomization, Corner Cases, Stress Testing, Coverage Metrics, Underflow, Overflow, Reset

Link al repositorio de git: https://github.com/Tommycampos07/fifo_verificacion.git (los archivos a revisar están en la carpeta “tb”)

Introducción

La verificación funcional es una fase crítica en el diseño de circuitos integrados, destinada a garantizar que el dispositivo cumpla con sus especificaciones antes de la fabricación. Este informe detalla la verificación de una memoria FIFO mediante un entorno modular desarrollado en SystemVerilog bajo el paradigma de programación orientada a objetos.

La arquitectura del ambiente integra componentes especializados como un *driver*, monitor y un *scoreboard* automatizado. Mediante el uso de aleatorización restringida y argumentos de tiempo de ejecución (*plusargs*), se ejecutaron diversos escenarios de prueba que incluyen casos esquina y condiciones de estrés. Esta metodología asegura una validación exhaustiva de la lógica de control, el manejo de datos y la sincronización del componente.

Device Under Testing (DUT)

La FIFO que vamos a verificar tiene las siguientes señales:

- **clk:** señal de reloj principal.
- **rst:** reset asíncrono.
- **push:** habilitación de escritura.
- **pop:** habilitación de lectura.
- **pndng:** “empty” me indica cuando la FIFO tiene un dato aún o está vacía.
- **full:** me indica cuando la FIFO ya no tiene espacios disponibles.

Ambiente de Verificación/Environment

Interfase

Encargado de contener todas las señales necesarias para la comunicación con el DUT, Esto genera un acceso uniforme a las entradas y salidas, lo que simplifica la conexión del driver y el monitor.

Transacción

Define las operaciones posibles de la FIFO (escritura, lectura y reset). Se incluyen señales tipo “rand” para poder aleatorizar la generación de diversos estímulos.

Driver

Encargado de recibir las transacciones y las “traduce” hacia la interfase.

- Entradas: agnt_drvr_mbx
- Salidas: vif.din, vif.push, vif.pop y vif.rst

Comportamiento: respeta las condiciones de la FIFO cuando está vacia o llena.
Eventos: reset, escritura y lectura.

Métricas: tiempo de respuesta, operaciones exitosas realizadas.

Monitor

Se encarga de observar la actividad entre el DUT y el Driver mediante la interfase y reconstruir dichas transacciones.

- Entradas: vif.clk, vif.dout, vif.pndng y vif.full
- Salidas: mon_sb_mbx

Comportamiento: no interfiere con el DUT, solo registra.
Eventos: deteccion de reset, escritura, lectura.
Métricas: observabilidad de todas las senales críticas.

Scoreboard

Memoria de referencia que lleva el control de todo lo que ha ocurrido en el sistema.

- Entradas: agt_sb_mbx , mon_sb_mbx , tst_sb_mbx

Comportamiento: utiliza una cola como modelo dorado para comparar datos escritos contra datos leídos.
Eventos: underflow, overflow, mismatches
Métricas: consistencia 100% entre entradas y salidas esperadas.

Generador

Se encarga de generar las secuencias de estímulos en base a los escenarios de prueba definido por los plusargumentos.

- Entradas: tr_mon.dato , ref_fifo
- Salidas: errores

Comportamiento: genera secuencias determinísticas o aleatorias.
Eventos: inicio de test, casos de esquina, rafagas de estres.
Métricas: cobertura de escenarios críticos y típicos.

Checker

Se encarga de validar todas las transacciones registradas en el scoreboard, con las observaciones del monitor, es el que decide si una transacción fue válida o no.

Environment/Ambiente

Integra todos los componentes previos: driver, monitor, transacción, interfase, scoreboard generador y checker. Y se encarga de coordinar la comunicación mediante mailboxes y eventos, asegurando sincronización al ejecutar el plan de verificación.

Pruebas definidas

Pruebas generales

Set de pruebas que prueban los distintos plusargumentos para verificar su correcto funcionamiento.
Escenarios: Prueba de overflow, prueba de baja latencia, prueba de underflow, prueba de reset con una seed específica, write/read simultáneo con alta latencia
Respuesta esperada: No se generan errores en las transacciones, no hay desbordes inesperados ni pérdidas, corrida exitosa.

Pruebas de Estrés

Centrado en ráfagas largas de operaciones para verificar la robustez del diseño.
Respuesta esperada: Se espera un tiempo de corrida promedio mayor al de las pruebas generales sin errores ni pérdidas en las transacciones esperadas.

Pruebas de Esquina

Enfocadas en validar las condiciones límite del diseño.
Respuesta esperada: Se espera que las señales full y pndng tienen el comportamiento adecuado, no hay corrupción en los datos.

Limitaciones

No se pueden implementar patrones de datos binarios críticos específicos como 0xFF, 0xAA y 0x00 ya que la generación de secuencias de datos es aleatoria por defecto.

La implementación se pensó de forma que cada bloque resolviera una parte bien específica del problema. El DUT de la FIFO se hizo con una memoria interna, dos punteros y un contador, porque esa es la forma más directa de modelar una cola real en hardware: un puntero escribe, otro lee, y el contador dice cuántos datos hay almacenados. A partir de ese contador se generan pndng y full, que son las señales que le dicen al resto del sistema si hay datos disponibles o si ya no cabe nada más. Dentro del always_ff se separaron los casos de escritura, lectura y lectura/escritura simultánea, porque cada uno afecta distinto el estado interno. Por ejemplo, en escritura sola se incrementa el contador si no está llena; en lectura sola se decrementa si hay datos pendientes; y en la operación simultánea se intenta conservar la ocupación mientras entra un dato nuevo y sale otro. Esa separación fue importante porque justo ahí están varios de los casos más delicados de una FIFO y era necesario que el comportamiento quedara explícito desde el código.

En el ambiente de verificación se siguió con la misma idea de “repartir responsabilidades”. Para la interfaz se usó para agrupar las señales del DUT y evitar estar pasando puertos por todo lado. Luego el driver toma una transacción y la convierte en señales reales sobre la interfaz, o sea, él es el que realmente activa push, pop, rst y pone din en el momento correcto. El monitor, por otro lado, observa lo que pasó en la interfaz y lo vuelve a empaquetar como transacción, para que el resto del ambiente no tenga que trabajar directamente con señales. Después de eso, el scoreboard mantiene una cola de referencia con el comportamiento esperado de la FIFO, mientras que el checker compara esa expectativa con lo que el monitor observó. Se podría decir que el scoreboard predice y el checker decide si hubo acierto o error. Esa fue la lógica principal de la implementación: separar aplicación, observación y validación y flujo de datos del testbench claro.

Por encima de eso se dejó el agent, el ambiente y el test como capa de control. El agent es el que genera las transacciones según el escenario que se quiera probar, ya sea aleatorio o dirigido, y de ahí salen tanto los casos básicos como los de esquina. El ambiente simplemente conecta todos los mailboxes e instancia cada componente para que corran en paralelo. Finalmente, el test quedó como el punto donde se configuran las corridas y se leen los plusargs, especialmente para los corner cases. Ahí fue donde se metieron modos como overflow, underflow, patrones de bits, simultaneidad en distintos niveles de llenado y reset en condiciones específicas. La idea detrás de implementarlo así fue no tener que estar reescribiendo el testbench cada vez que se quisiera cambiar de prueba, sino dejar una base fija y modificar solo el tipo de escenario que ejecuta el agent. Eso hace que se cumpla que el ambiente es reusable y también más cómodo para correr regresiones o probar varios casos desde línea de comandos.

El archivo fifo.sv corresponde al DUT y ahí se implementó la FIFO como una estructura secuencial sencilla pero bastante clara. La lógica se basa en una memoria interna mem, un puntero de escritura wr_ptr, un puntero de lectura rd_ptr y un contador count. Esa combinación permite modelar directamente el comportamiento de una cola FIFO en hardware: el puntero de escritura indica dónde entra el siguiente dato, el de lectura indica cuál es el siguiente dato que debe salir, y el contador permite saber en todo momento cuántos elementos hay almacenados. A partir de ese contador se generan las señales pndng y full, que simplifican mucho tanto la operación del DUT como su verificación, porque permiten saber si hay datos disponibles o si ya no se puede aceptar más información.

Dentro del bloque secuencial se separaron explícitamente los casos de solo escritura, solo lectura y lectura/escritura simultánea. Esa división no fue casual, sino que responde a la necesidad de dejar bien definido cómo cambia el estado interno en cada situación. En escritura se incrementa el contador si la FIFO no está llena; en lectura se decrementa si hay datos pendientes; y en la operación simultánea se maneja el caso de entrada y salida en el mismo ciclo, procurando conservar la ocupación cuando corresponde. También se cuidó el comportamiento del reset para limpiar punteros, contador y salida. Esta forma de implementarlo hace que el DUT sea fácil de seguir y, al mismo tiempo, que los casos importantes para verificación —overflow, underflow, reset y simultaneidad— queden claramente expuestos desde la lógica misma del diseño.

El archivo fifo_if.sv define la interfaz que agrupa todas las señales entre el DUT y el ambiente de verificación. Aunque es un archivo pequeño, cumple una función importante dentro de la implementación, porque evita que cada componente tenga que conectarse manualmente a señales individuales como clk, rst, push, pop, din, dout, pndng y full. En lugar de eso, toda esa comunicación queda centralizada en un solo punto, lo que hace que el testbench sea más ordenado y menos propenso a errores de conexión.

La decisión de usar una interfaz también ayuda a que driver, monitor y test trabajen sobre la misma vista del DUT, sin duplicar definiciones ni depender de listas largas de puertos. En la práctica, esto simplifica bastante el flujo del ambiente, porque cada componente solo necesita una referencia a la interfaz para poder leer o manejar las señales que le corresponden. Aunque su lógica interna es mínima, este archivo sirve como base de todo el intercambio entre diseño y verificación.

Este es el archivo más importante del ambiente, porque aquí se concentró prácticamente toda la lógica de verificación. En lugar de repartir clases y tipos en muchos archivos pequeños, se usó fifo_p.sv como paquete central para definir las transacciones, enumerados, mailboxes y cada uno de los componentes principales del ambiente. La idea detrás de eso fue tener en un solo lugar el “lenguaje” con el que trabajan todos los bloques: tipos de transacción, instrucciones del agente, solicitudes al scoreboard y clases que representan lo que entra, sale y se compara. Eso hace que el ambiente sea más fácil de mantener y también más fácil de entender, porque toda la estructura común queda reunida en un solo archivo.

Dentro de este paquete, la clase trans_fifo se implementó para representar cada operación que circula por el testbench. En vez de trabajar directamente con señales, se encapsularon campos como dato, retardo, tiempo y tipo, porque eso permite manejar la verificación a un nivel más alto de abstracción. También se incluyeron límites como max_retardo y min_retardo, que luego se usan para controlar la aleatorización. Esta parte fue importante porque permitió que el mismo ambiente pudiera usarse tanto para pruebas generales como para corner cases, sin tener que rehacer la estructura base cada vez.

Cabe recalcar también que los mailboxes se usaron como mecanismo principal de comunicación entre los componentes del ambiente. La lógica detrás de esto fue desacoplar cada bloque, de manera que el driver, monitor, checker, scoreboard, agent y test no dependieran de llamadas directas ni de una ejecución estrictamente secuencial. En vez de eso, cada componente recibe o envía transacciones por su respectivo mailbox, lo que

permite que todos corran en paralelo y que la información fluya de forma ordenada entre generación, aplicación, observación y comparación. Esta implementación hace que el ambiente sea modular, refiriéndose a que ya cada bloque solo necesita conocer su canal de entrada y salida, no la lógica interna de los demás.

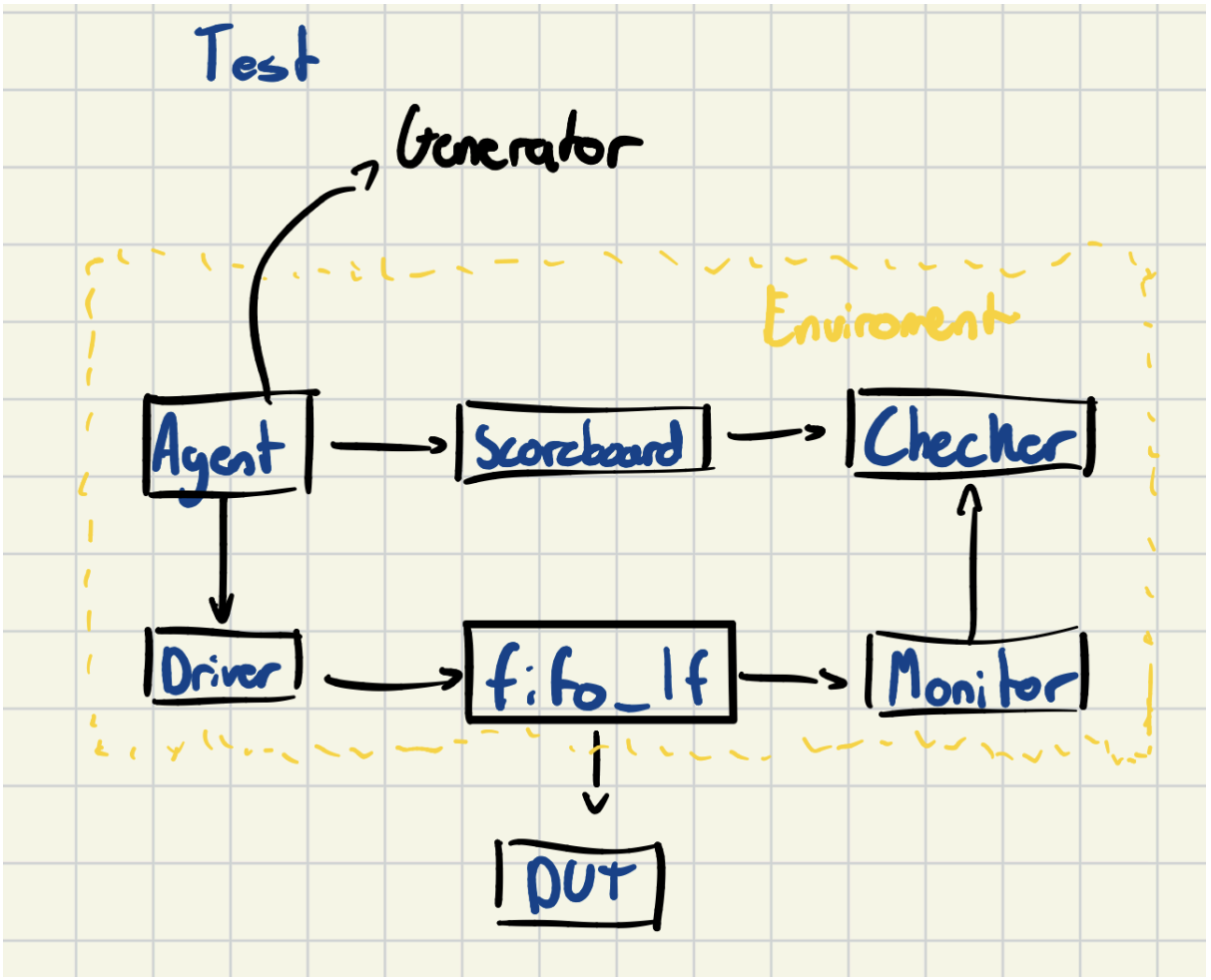

Fig. 1 Diagrama de ambiente

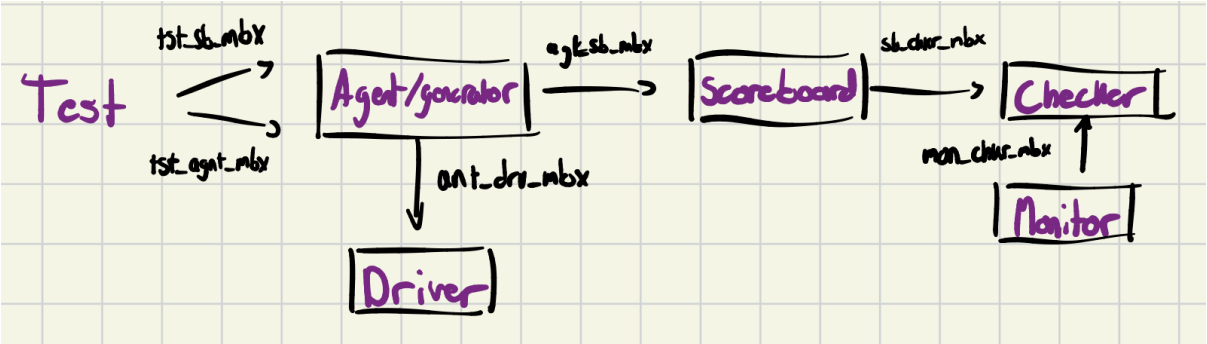

Fig. 2 Diagrama de mailboxes

Resultados:

Durante la ejecución del plan de verificación, se espera que las simulaciones de la FIFO reflejen el comportamiento definido en las especificaciones funcionales. Los resultados se analizan principalmente mediante el scoreboard, que valida la correspondencia entre los datos escritos y leídos.

- La señal full debe activarse al alcanzar la capacidad máxima de la FIFO. La señal pndng debe desactivarse únicamente cuando no existan datos almacenados. En condiciones de overflow (escribir cuando full=1), los datos no deben sobrescribirse. En condiciones de underflow (leer cuando la FIFO está vacía), no deben generarse datos inválidos.
- La secuencia de lecturas debe coincidir con la secuencia de escrituras según el principio FIFO. No deben existir pérdidas, duplicaciones ni alteraciones de datos.

```
[CRUZ_JUAN@redhat003 tb]$ source comando.sh
                         Chronologic VCS (TM)
       Version R-2020.12-SP2_Full64 -- Tue Apr 28 23:03:47 2026

                    Copyright (c) 1991 - 2021 Synopsys, Inc.
   This software and the associated documentation are proprietary to Synopsys,
 Inc. This software may only be used in accordance with the terms and conditio
ns
 of a written license agreement with Synopsys, Inc. All other use, reproductio
n,
            or distribution of this software is strictly prohibited.


Warning-[LCA_FEATURES_ENABLED] Usage warning
  LCA features enabled by '-lca' argument on the command line.  For more
  information regarding list of LCA features please refer to Chapter "LCA
  features" in the VCS Release Notes

The design hasn't changed and need not be recompiled.
If you really want to, delete file /mnt/vol_NFS_rh003/Est_Veri_I_2026/CRUZ_JUA
N/proyecto1/fifo_verificacion/tb/salida.daidir/.vcs.timestamp and
run VCS again.

Chronologic VCS simulator copyright 1991-2020
Contains Synopsys proprietary information.
Compiler version R-2020.12-SP2_Full64; Runtime version R-2020.12-SP2_Full64;
Apr 28 23:03 2026
╔═══════════════════════════════╗
║        CONFIGURACION DEL TEST
╟───────────────────────────────
  MODO          = SIM_ALTO
  NUM_TRANS     = 20
  MAX_RETARDO   = 0
  MIN_RETARDO   = 0
  SEED          = 777
╚═══════════════════════════════╝
[0] Test iniciado — modo: SIM_ALTO
```

```
========== REPORTE SCOREBOARD ==========
Reportes generados        = 1
Transacciones esperadas   = 26
========================================

[Monitor envia] tiempo=1710 tipo=3 retardo=0 dato=0x50

[1710000] Checker compara transaccion #23
[Esperada] tiempo=35 tipo=3 retardo=0 dato=0x50
[Observada] tiempo=1710 tipo=3 retardo=0 dato=0x50
[1710000] Checker PASS: lectura_escritura correcta
[Monitor envia] tiempo=1730 tipo=3 retardo=0 dato=0x6c

[1730000] Checker compara transaccion #24
[Esperada] tiempo=35 tipo=3 retardo=0 dato=0x6c
[Observada] tiempo=1730 tipo=3 retardo=0 dato=0x6c
[1730000] Checker PASS: lectura_escritura correcta
[1755000] Test: fin
$finish called from file "fifo_p.sv", line 1198.
$finish at simulation time              1755000
           V C S   S i m u l a t i o n   R e p o r t
Time: 1755000 ps
CPU Time:      0.920 seconds;       Data structure size:   0.0Mb
```

Figura 3: Prueba general de alta latencia

```
Chronologic VCS simulator copyright 1991-2020
Contains Synopsys proprietary information.
Compiler version R-2020.12-SP2_Full64; Runtime version R-2020.12-SP2_Full64;
Apr 28 23:08 2026

╔═══════════════════════════════════════╗
║        CONFIGURACION DEL TEST         ╒
╚═══════════════════════════════════════╝
  MODO          = SIM_MEDIO
  NUM_TRANS     = 150
  MAX_RETARDO   = 3
  MIN_RETARDO   = 1
  SEED          = 42
╚═══════════════════════════════════════╝
[0] Test iniciado — modo: SIM_MEDIO
[0] Ambiente inicializado
[0] Checker inicializado
[0] Scoreboard inicializado
[0] Agent inicializado
[35000] Test: simultaneo_medio
[Agent envia] tiempo=35 tipo=1 retardo=2 dato=0x73
[Agent envia] tiempo=35 tipo=1 retardo=9 dato=0x1e
[Agent envia] tiempo=35 tipo=1 retardo=0 dato=0x14
[Agent envia] tiempo=35 tipo=1 retardo=6 dato=0x6c
[Agent envia] tiempo=35 tipo=3 retardo=7 dato=0x3b
```

```
[10210000] Checker compara transaccion #152
[Esperada] tiempo=35 tipo=3 retardo=0 dato=0x18
[Observada] tiempo=10210 tipo=3 retardo=0 dato=0x18
[10210000] Checker PASS: lectura_escritura correcta
[Monitor envia] tiempo=10270 tipo=3 retardo=0 dato=0x10

[10270000] Checker compara transaccion #153
[Esperada] tiempo=35 tipo=3 retardo=0 dato=0x10
[Observada] tiempo=10270 tipo=3 retardo=0 dato=0x10
[10270000] Checker PASS: lectura_escritura correcta
[Monitor envia] tiempo=10300 tipo=3 retardo=0 dato=0x4c

[10300000] Checker compara transaccion #154
[Esperada] tiempo=35 tipo=3 retardo=0 dato=0x4c
[Observada] tiempo=10300 tipo=3 retardo=0 dato=0x4c
[10300000] Checker PASS: lectura_escritura correcta
[11595000] Test: reporte solicitado al scoreboard

========== REPORTE SCOREBOARD ==========
Reportes generados        = 1
Transacciones esperadas   = 154
========================================

[11695000] Test: fin
$finish called from file "fifo_p.sv", line 1198.
$finish at simulation time             11695000
           V C S   S i m u l a t i o n   R e p o r t
Time: 11695000 ps
CPU Time:      0.920 seconds;       Data structure size:   0.0Mb
```

Figura 4: Caso esquina donde se levantan las banderas full y pndng constantemente

```
Chronologic VCS simulator copyright 1991-2020
Contains Synopsys proprietary information.
Compiler version R-2020.12-SP2_Full64; Runtime version R-2020.12-SP2_Full64;
Apr 28 23:14 2026

╔═══════════════════════════════════╗
║        CONFIGURACION DEL TEST     ╒
╚═══════════════════════════════════╝
  MODO          = SIM_BAJO
  NUM_TRANS     = 800
  MAX_RETARDO   = 15
  MIN_RETARDO   = 5
  SEED          = aleatorio

[0] Test iniciado — modo: SIM_BAJO
[0] Ambiente inicializado
[0] Checker inicializado
[0] Scoreboard inicializado
[0] Agent inicializado
[35000] Test: simultaneo_bajo
```

```
[55710000] Checker compara transaccion #802
[Esperada] tiempo=35 tipo=3 retardo=0 dato=0xbb
[Observada] tiempo=55710 tipo=3 retardo=0 dato=0xbb
[55710000] Checker PASS: lectura_escritura correcta
[154055000] Test: reporte solicitado al scoreboard

========== REPORTE SCOREBOARD ==========
Reportes generados        = 1
Transacciones esperadas   = 802
========================================

[154155000] Test: fin
$finish called from file "fifo_p.sv", line 1198.
$finish at simulation time            154155000
           V C S   S i m u l a t i o n   R e p o r t
Time: 154155000 ps
CPU Time:      1.070 seconds;       Data structure size:   0.0Mb
Tue Apr 28 23:14:11 2026
[CRUZ_JUAN@redhat003 tb]$
```

Figura 5: Prueba de estrés con underflow

Conclusiones

Tras completar el plan de verificación, se confirma que la memoria FIFO opera de manera robusta y adecuada bajo todas las condiciones evaluadas. Las pruebas de estrés validaron la precisión de las banderas de estado y la estabilidad de los punteros internos, incluso durante operaciones simultáneas de alta velocidad.

El uso de semillas aleatorias y la inyección de patrones de datos específicos permitieron alcanzar una cobertura funcional completa, descartando errores de latencia o corrupción de información. En definitiva, los resultados obtenidos a través del *scoreboard* demuestran que el diseño es fiable y está listo para su integración en sistemas digitales de mayor escala.